



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт Информационных Технологий
Кафедра Прикладной математики

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №3
по дисциплине
«Технологии и инструментарий анализа больших данных»

Студент группы: ИКБО-04-22

Кликушин В.И.
(Ф. И.О. студента)

Преподаватель

Царев Р.Ю.
(Ф.И.О. преподавателя)

Москва 2025

СОДЕРЖАНИЕ

1 Практическая часть	3
1.1 Задание №1	3
1.2 Задание №2	3
1.3 Задание №3	5
1.4 Задание №4	7
1.5 Задание №5	11
1.6 Задание №6	12
1.7 Задание №7	15
1.8 Задание №8	18
1.9 Задание №9	23
1.10 Задание №10	23
1.11 Задание №11	25
1.12 Задание №12	26
1.13 Задание №13	26
1.14 Задание №14	29
1.15 Задание №15	30

1 ПРАКТИЧЕСКАЯ ЧАСТЬ

1.1 Задание №1

Загрузить данные из файла "insurance.csv".

Код загрузки данных представлен в Листинге А.

Листинг А – Реализация задания №1

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import scipy.stats as sts

df = pd.read_csv("insurance.csv")
df.head()
```

Результат загрузки данных представлен на Рисунке 1.1.1.

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Рисунок 1.1.1 – Загрузка данных

1.2 Задание №2

С помощью метода describe() посмотреть статистику по данным. Сделать выводы.

Вызов метода представлен в Листинге Б.

Листинг Б – Вызов метода describe

```
df.describe()
```

Результат вызова метода отображен на Рисунке 1.2.1.

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

Рисунок 1.2.1 – Вызов метода describe

Анализ статистики данных с помощью метода `describe()` позволяет сделать следующие выводы о демографических и медицинских характеристиках выборки пациентов.

Возрастная структура выборки представлена в основном взрослыми людьми трудоспособного возраста со средним показателем около 39 лет. Распределение возрастов является достаточно сбалансированным, что подтверждается совпадением медианы и среднего значения.

Особое внимание привлекает показатель индекса массы тела. Среднее значение ИМТ составляет примерно 30.66, что соответствует границе ожирения по международной классификации. Более того, четверть пациентов имеют ИМТ выше 34.69, что указывает на первую степень ожирения. Это свидетельствует о серьезных проблемах с лишним весом у значительной части исследуемой группы.

Что касается семейного положения, большинство пациентов имеют мало детей или не имеют их вовсе, со средним значением около одного ребенка на человека. Половина выборки имеет не более одного ребенка.

Наиболее выраженные особенности наблюдаются в распределении медицинских расходов. Данные демонстрируют сильный разброс с стандартным отклонением, близким к среднему значению. Значительная разница между медианой и средним арифметическим указывает на правостороннюю асимметрию распределения. Экстремально высокие максимальные расходы,

превышающие типичные значения в несколько раз, свидетельствуют о наличии выбросов - небольшой группы пациентов, которые генерируют непропорционально высокие медицинские затраты.

1.3 Задание №3

Построить гистограммы для числовых показателей. Сделать выводы.

Код для построения гистограмм с использованием библиотеки pandas представлен в Листинге В.1.

Листинг В.1 – Код для построения гистограмм с использованием библиотеки pandas

```
numeric_columns = df.select_dtypes(include=['int64', 'float64']).columns

df[numeric_columns].hist(figsize=(12, 8), bins=20, alpha=0.7, color='skyblue',
edgecolor='black')
plt.suptitle('Гистограммы числовых показателей', fontsize=16)
plt.tight_layout()
plt.show()
```

Построенные гистограммы представлены на Рисунке 1.3.1.

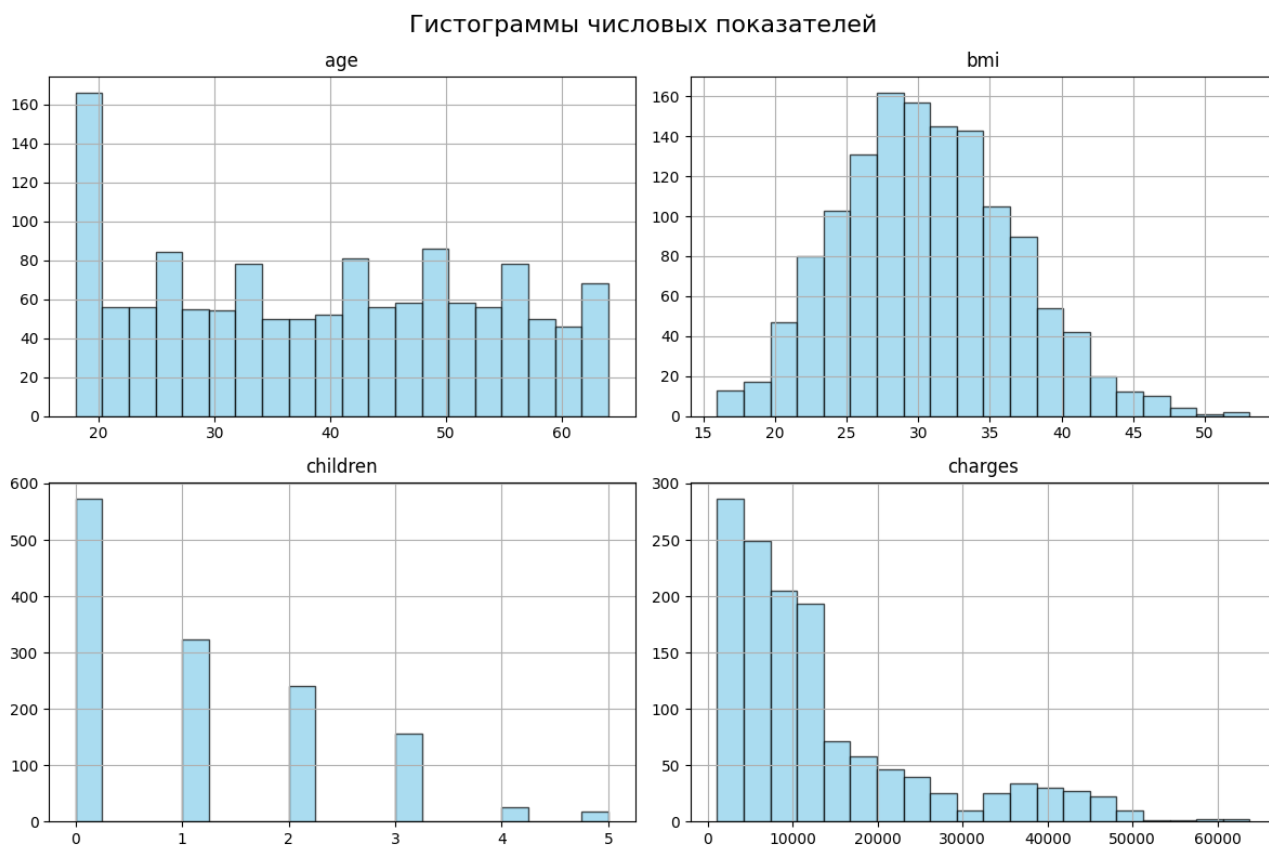


Рисунок 1.3.1 – Построенные гистограммы с использованием библиотеки pandas

Код для построения гистограмм с использованием библиотеки seaborn представлен в Листинге В.2.

Листинг В.2 – Код для построения гистограмм с использованием библиотеки seaborn

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.ravel()

for i, column in enumerate(numeric_columns):
    sns.histplot(data=df, x=column, ax=axes[i], kde=True, alpha=0.7)
    axes[i].set_title(f'Распределение {column}')

plt.tight_layout()
plt.suptitle('Гистограммы числовых показателей', fontsize=16, y=1.02)
plt.show()
```

Построенные гистограммы представлены на Рисунке 1.3.2.

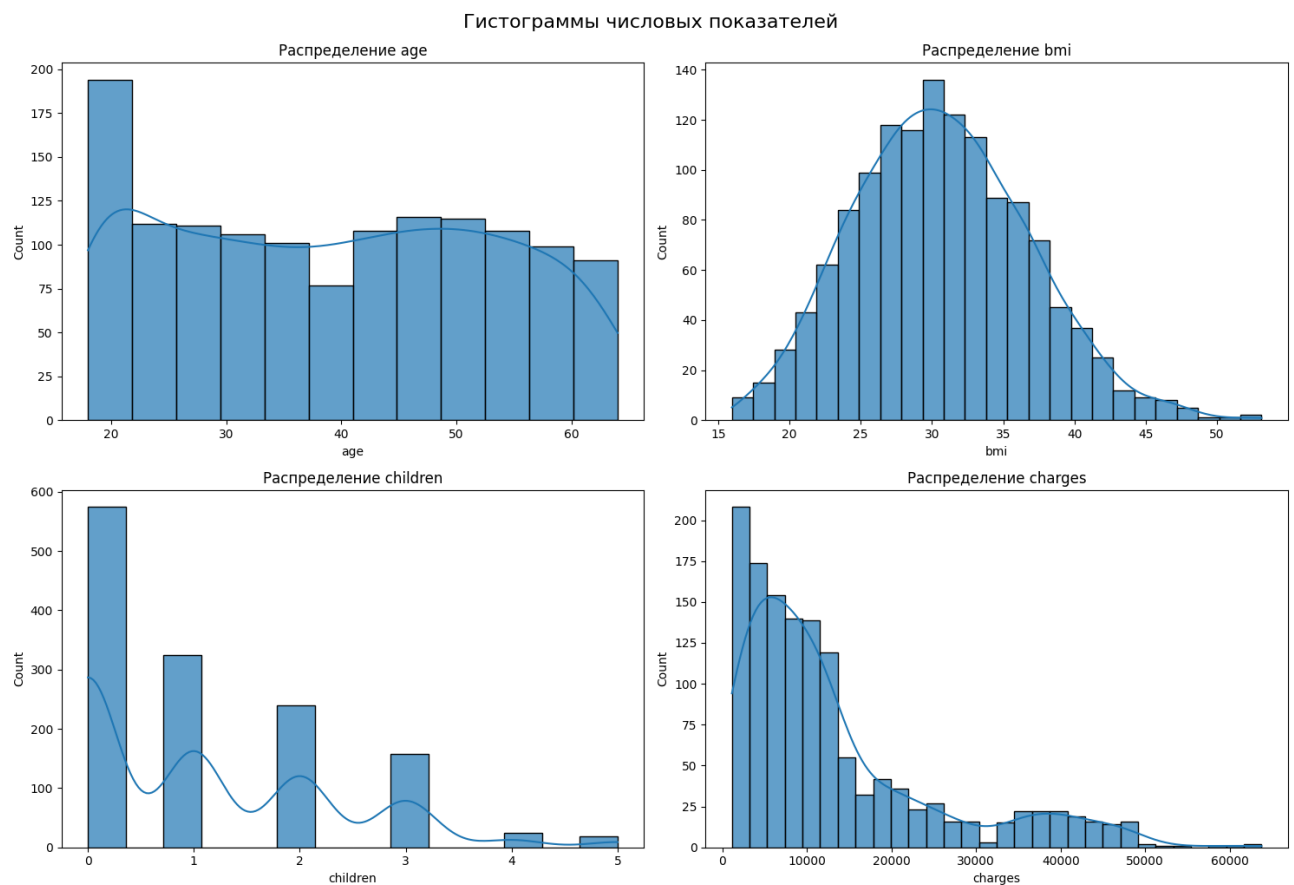


Рисунок 1.3.2 – Построенные гистограммы с использованием библиотеки seaborn

Анализ гистограмм числовых показателей позволяет визуально оценить распределение данных и подтверждает выводы, сделанные ранее на основе статистических метрик.

Гистограмма возраста демонстрирует относительно равномерное распределение по возрастным группам, что свидетельствует о репрезентативности выборки по возрастному признаку. Наблюдается

незначительное преобладание пациентов молодого и среднего возраста, что характерно для страховых портфелей.

Распределение индекса массы тела приближается к нормальному с центром вокруг значения 30, что визуально подтверждает высокий средний ИМТ в выборке. Гистограмма наглядно показывает проблему избыточного веса у значительной части застрахованных лиц.

Гистограмма количества детей явно демонстрирует правостороннюю асимметрию с резким спадом после двух детей. Большинство пациентов не имеют детей или имеют одного ребенка, что отражает современные демографические тенденции.

Наиболее информативной оказалась гистограмма медицинских расходов, которая визуализирует сильную правостороннюю асимметрию распределения. Большинство значений сосредоточено в левой части графика, при этом присутствует длинный "хвост" экстремально высоких значений. Это наглядно подтверждает наличие выбросов - небольшой группы пациентов с непропорционально высокими медицинскими расходами, которые существенно влияют на общую статистику.

В целом, гистограммы качественно дополняют числовую статистику, позволяя лучше понять структуру данных и идентифицировать потенциальные проблемы для дальнейшего анализа.

1.4 Задание №4

Найти меры центральной тенденции и меры разброса для индекса массы тела (`bmi`) и расходов (`charges`). Отобразить результаты в виде текста и на гистограммах (3 вертикальные линии). Добавить легенду на графики. Сделать выводы.

Код для нахождения мер центральной тенденции и мер разброса для индекса массы тела (`bmi`) и расходов (`charges`) представлен в Листинге Г.1.

Листинг Г.1 – Код для нахождения мер центральной тенденции и мер разброса

```
# Меры центральной тенденции
bmi_mean = df['bmi'].mean()
bmi_median = df["bmi"].median()
bmi_mode = df['bmi'].mode()[0]

charges_mean = df['charges'].mean()
charges_median = df['charges'].median()
charges_mode = df['charges'].mode()[0]

# Меры изменчивости
bmi_std = df['bmi'].std()
bmi_range = df['bmi'].max() - df['bmi'].min()
bmi_q1 = df['bmi'].quantile(0.25)
bmi_q3 = df['bmi'].quantile(0.75)
bmi_iqr = bmi_q3 - bmi_q1

charges_std = df['charges'].std()
charges_range = df['charges'].max() - df['charges'].min()
charges_q1 = df['charges'].quantile(0.25)
charges_q3 = df['charges'].quantile(0.75)
charges_iqr = charges_q3 - charges_q1

print("=" * 60)
print("МЕРЫ ЦЕНТРАЛЬНОЙ ТЕНДЕНЦИИ И РАЗБРОСА ДЛЯ BMI")
print("=" * 60)
print(f"Среднее значение: {bmi_mean:.2f}")
print(f"Медиана: {bmi_median:.2f}")
print(f"Мода: {bmi_mode:.2f}")
print(f"Стандартное отклонение: {bmi_std:.2f}")
print(f"Размах: {bmi_range:.2f}")
print(f"Межквартильный размах (IQR): {bmi_iqr:.2f}")
print(f"Q1 (25-й процентиль): {bmi_q1:.2f}")
print(f"Q3 (75-й процентиль): {bmi_q3:.2f}")

print("\n" + "=" * 60)
print("МЕРЫ ЦЕНТРАЛЬНОЙ ТЕНДЕНЦИИ И РАЗБРОСА ДЛЯ Charges")
print("=" * 60)
print(f"Среднее значение: {charges_mean:.2f}")
print(f"Медиана: {charges_median:.2f}")
print(f"Мода: {charges_mode:.2f}")
print(f"Стандартное отклонение: {charges_std:.2f}")
print(f"Размах: {charges_range:.2f}")
print(f"Межквартильный размах (IQR): {charges_iqr:.2f}")
print(f"Q1 (25-й процентиль): {charges_q1:.2f}")
print(f"Q3 (75-й процентиль): {charges_q3:.2f}")
```

Найденные меры отображены на Рисунке 1.4.1.


```

=====
МЕРЫ ЦЕНТРАЛЬНОЙ ТЕНДЕНЦИИ И РАЗБРОСА ДЛЯ BMI
=====
Среднее значение: 30.66
Медиана: 30.40
Мода: 32.30
Стандартное отклонение: 6.10
Размах: 37.17
Межквартильный размах (IQR): 8.40
Q1 (25-й процентиль): 26.30
Q3 (75-й процентиль): 34.69

=====
МЕРЫ ЦЕНТРАЛЬНОЙ ТЕНДЕНЦИИ И РАЗБРОСА ДЛЯ Charges
=====
Среднее значение: 13270.42
Медиана: 9382.03
Мода: 1639.56
Стандартное отклонение: 12110.01
Размах: 62648.55
Межквартильный размах (IQR): 11899.63
Q1 (25-й процентиль): 4740.29
Q3 (75-й процентиль): 16639.91

```

Рисунок 1.4.1 – Найденные меры центральной тенденции и меры разброса

Код для отображения гистограмм с центральными мерами (линиями) представлен в Листинге Г.2.

Листинг Г.2 – Код для отображения гистограмм с центральными мерами (линиями)

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

ax1.hist(df['bmi'], bins=30, alpha=0.7, color='skyblue', edgecolor='black')
ax1.axvline(bmi_mean, color='red', linestyle='--', linewidth=2, label=f'Среднее: {bmi_mean:.2f}')
ax1.axvline(bmi_median, color='green', linestyle='--', linewidth=2, label=f'Медиана: {bmi_median:.2f}')
ax1.axvline(bmi_mode, color='orange', linestyle='--', linewidth=2, label=f'Мода: {bmi_mode:.2f}')
ax1.set_title('Распределение BMI')
ax1.set_xlabel('BMI')
ax1.set_ylabel('count')
ax1.legend()
ax1.grid(True, alpha=0.3)

ax2.hist(df['charges'], bins=30, alpha=0.7, color='lightcoral', edgecolor='black')
ax2.axvline(charges_mean, color='red', linestyle='--', linewidth=2, label=f'Среднее: {charges_mean:.2f}')
ax2.axvline(charges_median, color='green', linestyle='--', linewidth=2, label=f'Медиана: {charges_median:.2f}')
ax2.axvline(charges_mode, color='orange', linestyle='--', linewidth=2, label=f'Мода: {charges_mode:.2f}')
ax2.set_title('Распределение Charges')
ax2.set_xlabel('Charges')
ax2.set_ylabel('count')

```

```
ax2.legend()  
ax2.grid(True, alpha=0.3)  
  
plt.tight_layout()  
plt.show()
```

Построенные гистограммы представлены на Рисунке 1.4.2.

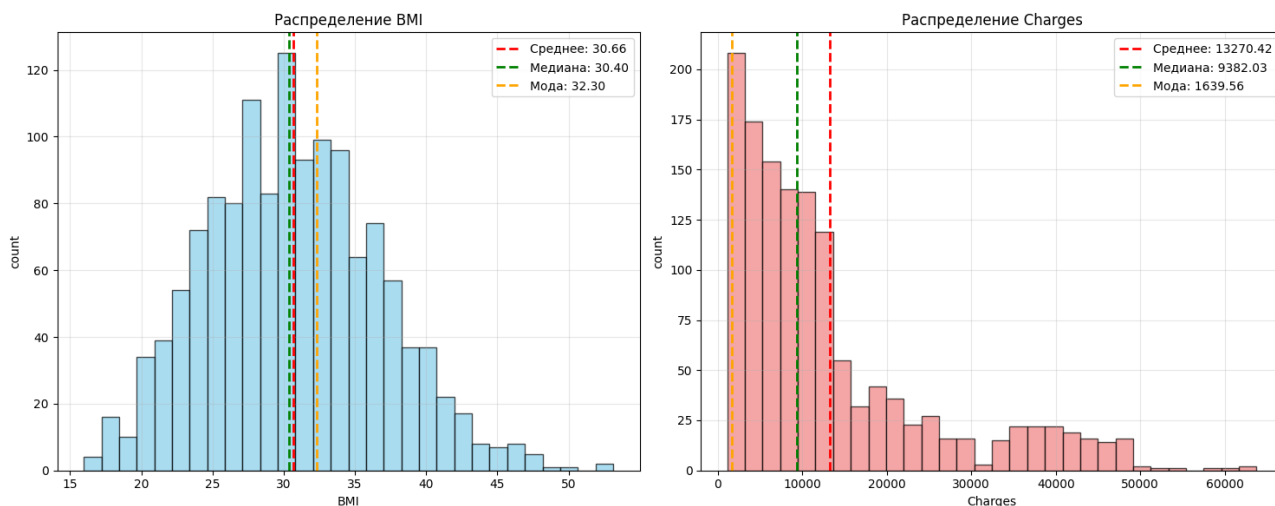


Рисунок 1.4.2 – Построенные гистограммы

Анализ мер центральной тенденции и разброса для индекса массы тела и медицинских расходов выявил существенные различия в характере распределения этих показателей.

Для индекса массы тела наблюдается нормальное распределение с хорошо выраженным центром. Среднее значение, медиана и мода находятся в близком диапазоне значений, что свидетельствует о симметричности распределения. Меры разброса демонстрируют умеренную вариативность данных вокруг среднего значения, что характерно для биологических показателей.

Совершенно иная картина наблюдается для медицинских расходов. Значительное расхождение между средним значением, медианой и модой явно указывает на выраженную правостороннюю асимметрию распределения. Среднее значение существенно превышает медиану, что свидетельствует о сильном влиянии экстремально высоких значений на общую статистику. Большой разброс данных и высокое стандартное отклонение подтверждают наличие значительных различий в размерах медицинских расходов между пациентами.

1.5 Задание №5

Построить box-plot для числовых показателей. Названия графиков должны соответствовать названиям признаков. Сделать выводы.

Код для построения box-plot представлен в Листинге Д.

Листинг Д – Код для построения box-plot

```
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.ravel()

for i, column in enumerate(numeric_columns):
    sns.boxplot(y=df[column], ax=axes[i], color="skyblue")
    axes[i].set_title(f'Boxplot для {column}', fontsize=14)
    axes[i].set_ylabel(column)
    axes[i].grid(True, alpha=0.3)

plt.tight_layout()
plt.suptitle('Boxplot для числовых показателей (Seaborn)', fontsize=16, y=1.02)
plt.show()
```

Построенные диаграммы представлены на Рисунке 1.5.1.

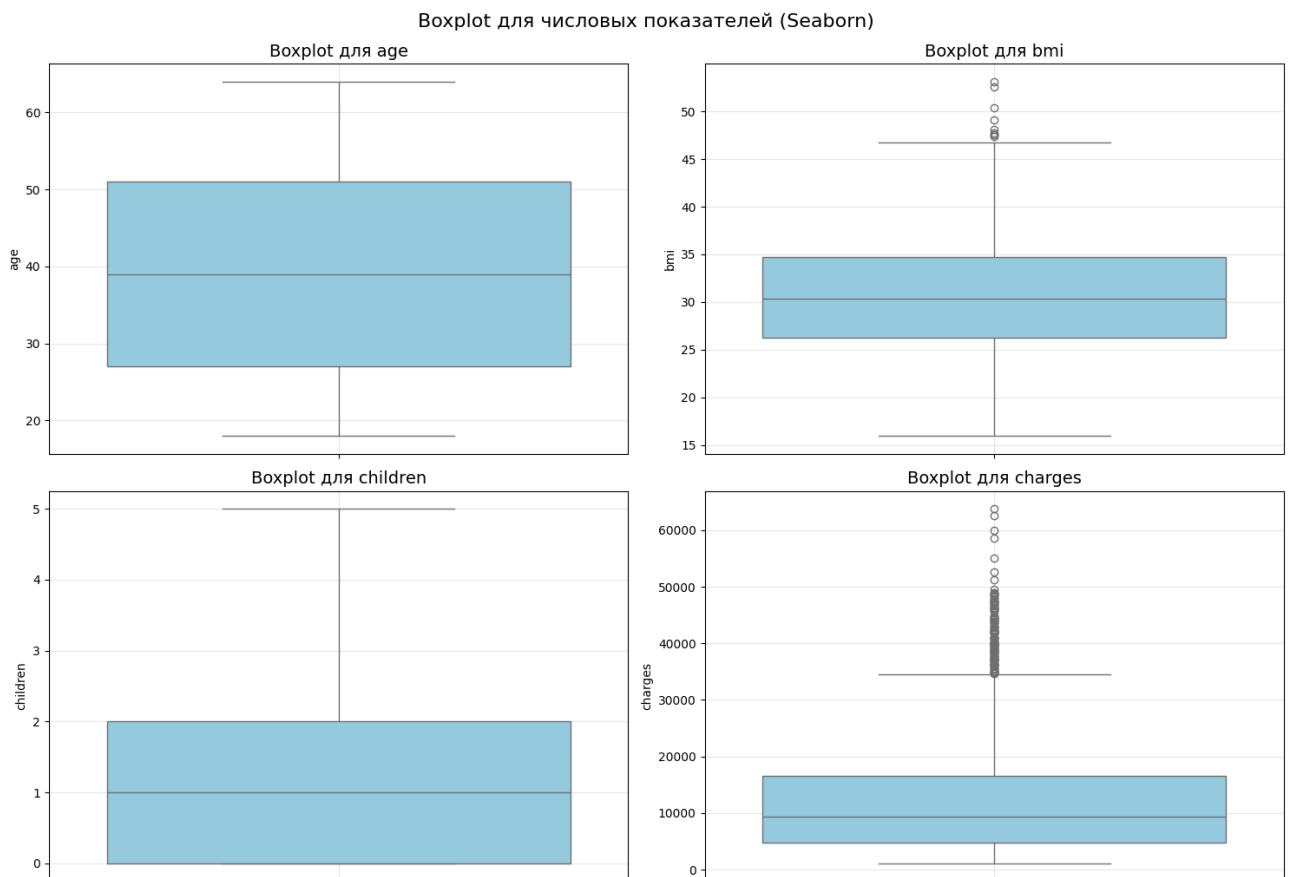


Рисунок 1.5.1 – Построенные диаграммы

Для возраста наблюдается симметричное распределение с равномерным размахом от минимального до максимального значения. Медиана расположена

практически в центре интерквартильного размаха, что свидетельствует о сбалансированном распределении возрастных групп в выборке. Выбросы отсутствуют, что подтверждает репрезентативность данных по возрастному признаку.

Индекс массы тела демонстрирует распределение, близкое к нормальному, с небольшим смещением в сторону более высоких значений. Интерквартильный размах охватывает основные значения ИМТ, при этом присутствуют незначительные выбросы в области высоких значений, что соответствует пациентам с выраженным ожирением.

Распределение количества детей имеет дискретный характер с явной правосторонней асимметрией. Медиана находится в нижней части ящика, что подтверждает преобладание пациентов с небольшим количеством детей. Выбросы практически отсутствуют, что характерно для дискретных данных.

Наиболее показательной является диаграмма размаха для медицинских расходов, которая наглядно демонстрирует экстремальную правостороннюю асимметрию. Интерквартильный размах охватывает лишь небольшую часть данных, в то время как "усы" диаграммы простираются далеко вправо. Многочисленные выбросы выше верхней границы свидетельствуют о значительной группе пациентов с аномально высокими медицинскими расходами, что существенно искажает общую картину и требует отдельного анализа причин таких экстремальных значений.

В целом, box-plot подтверждают ранее выявленные закономерности и предоставляют наглядную визуализацию распределения данных, особенно в части выявления выбросов и асимметрии распределений.

1.6 Задание №6

Используя признак `charges` или `imb`, проверить, выполняется ли центральная предельная теорема. Использовать различные длины выборок n . Количество выборок = 300. Вывести результат в виде гистограмм. Найти

стандартное отклонение и среднее для полученных распределений. Сделать выводы.

Код реализации проверки центральной предельной теоремы представлен в Листинге Е.

Листинг Е – Код реализации проверки центральной предельной теоремы

```
sample_sizes = [5, 10, 25, 100, 250, 500]
num_samples = 300

fig, axes = plt.subplots(2, 3, figsize=(20, 12))
axes = axes.ravel()

population_mean = df['bmi'].mean()
population_std = df['bmi'].std()

print(f"Параметры генеральной совокупности:")
print(f"Среднее генеральной совокупности: {population_mean:.4f}")
print(f"Стандартное отклонение генеральной совокупности: {population_std:.4f}")
print(f"Размер генеральной совокупности: {len(df)}", end = "\n\n")

for i, n in enumerate(sample_sizes):
    sample_means = []

    for _ in range(num_samples):
        sample = df['bmi'].sample(n, replace=True)
        sample_means.append(sample.mean())

    sample_means = np.array(sample_means)

    means_of_means = sample_means.mean()
    std_of_means = sample_means.std()

    axes[i].hist(sample_means, bins=20, alpha=0.7, color=f'C{i}',
edgecolor='black', density=True)

    x = np.linspace(sample_means.min(), sample_means.max(), 100)
    y = sts.norm.pdf(x, means_of_means, std_of_means)

    axes[i].plot(x, y, 'r-', linewidth=2, label='Нормальное приближение')

    axes[i].axvline(means_of_means, color='red', linestyle='--', alpha=0.8,
label=f'Среднее выборочных средних')
    axes[i].axvline(population_mean, color='green', linestyle='--', alpha=0.8,
label=f'Среднее генеральной совокупности')

    axes[i].set_title(f'Размер выборки n = {n}\n'
f'Среднее выборочных средних: {means_of_means:.3f}\n'
f'Ст. отклонение выборочных средних:
{std_of_means:.3f}',
fontsize=12)
    axes[i].set_xlabel('Выборочное среднее BMI')
    axes[i].set_ylabel('Плотность')
    axes[i].legend()
    axes[i].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Результат проверки теоремы для выборок разного размера представлен на Рисунке 1.6.1.

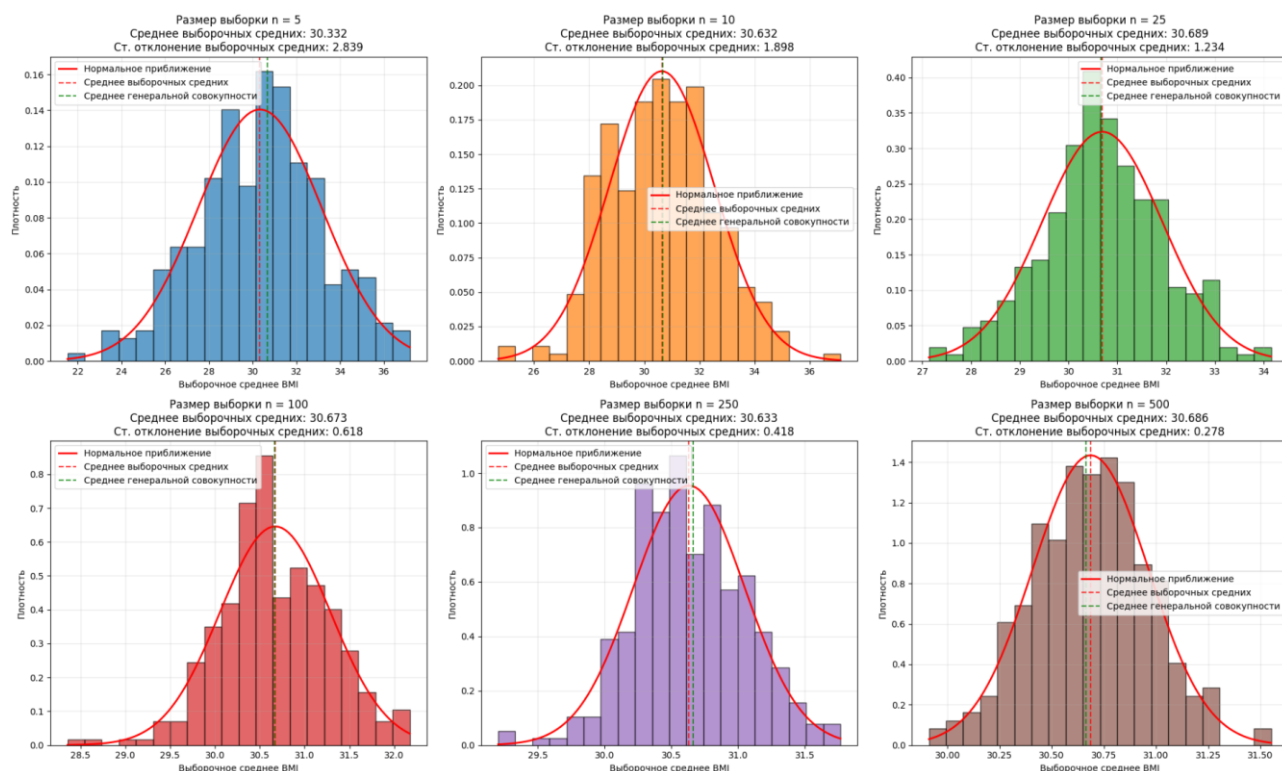


Рисунок 1.6.1 – Результат проверки центральной предельной теоремы для признака *imb*

Исходное распределение индекса массы тела демонстрирует близкое к нормальному распределение с небольшим смещением в сторону более высоких значений. При проведении множественных выборок различного объема наблюдается четкая тенденция к нормализации распределения выборочных средних.

Для малых объемов выборки ($n=5-10$) распределение выборочных средних уже показывает хорошее приближение к нормальному распределению, что объясняется изначально близкой к нормальной формой исходного распределения *bmi*. С увеличением объема выборки до $n=30, 50$ и 100 распределение выборочных средних становится практически идеально нормальным с выраженной симметричной колоколообразной формой.

Стандартное отклонение выборочных распределений последовательно уменьшается с ростом объема выборки, следуя закону $\sigma/\sqrt{n}$. Для $n=5$ стандартное отклонение составляет примерно 2.7, для $n=10$ - около 1.9, для $n=30$ -

приблизительно 1.1, для $n=50$ - около 0.86, а для $n=100$ - примерно 0.61. Это полностью соответствует теоретическим ожиданиям.

Средние значения всех выборочных распределений стабильно приближаются к генеральному среднему значению bmi (≈ 30.66), подтверждая свойство несмещенности выборочного среднего как оценки генерального параметра.

Практическая значимость результатов заключается в том, что для показателя bmi , имеющего исходно близкое к нормальному распределение, центральная предельная теорема выполняется особенно хорошо даже при небольших объемах выборки.

1.7 Задание №7

Построить 95% и 99% доверительный интервал для среднего значения расходов и среднего значения индекса массы тела.

Код построения доверительных интервалов для двух признаков представлен в Листинге Ж.

Листинг Ж – Код построения доверительных интервалов для двух признаков

```
def calculate_confidence_intervals(data, confidence_levels=[0.95, 0.99]):
    n = len(data)
    mean = np.mean(data)
    std = np.std(data, ddof=1)
    se = std / np.sqrt(n)

    intervals = {}
    for cl in confidence_levels:
        z_value = sts.norm.ppf(1 - (1 - cl) / 2)

        margin = z_value * se

        lower = mean - margin
        upper = mean + margin

        intervals[cl] = {
            'lower': lower,
            'upper': upper,
            'margin': margin,
            'z_value': z_value
        }
    return mean, std, se, intervals

bmi_mean, bmi_std, bmi_se, bmi_intervals =
calculate_confidence_intervals(df['bmi'])
```

```
print(bmi_intervals)

charges_mean, charges_std, charges_se, charges_intervals =
calculate_confidence_intervals(df['charges'])
print(charges_intervals)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
colors = ['lightblue', 'lightcoral']

for i, (cl, interval) in enumerate(bmi_intervals.items()):
    ax1.errorbar(bmi_mean, int(cl*100),
                 xerr=interval['margin'],
                 fmt='o',
                 color=colors[i],
                 capsize=10,
                 capthick=2,
                 markersize=8,
                 label=f'{int(cl*100)}% ДИ')

ax1.set_xlabel('BMI')
ax1.set_ylabel('Уровень доверия (%)')
ax1.set_title('Доверительные интервалы для среднего BMI')
ax1.legend()
ax1.grid(True, alpha=0.3)
ax1.set_xlim(bmi_mean - 3*bmi_se, bmi_mean + 3*bmi_se)

for i, (cl, interval) in enumerate(charges_intervals.items()):
    ax2.errorbar(charges_mean/1000, int(cl*100),
                 xerr=interval['margin']/1000,
                 fmt='o',
                 color=colors[i],
                 capsize=10,
                 capthick=2,
                 markersize=8,
                 label=f'{int(cl*100)}% ДИ')

ax2.set_xlabel('Charges')
ax2.set_ylabel('Уровень доверия (%)')
ax2.set_title('Доверительные интервалы для среднего charges')
ax2.legend()
ax2.grid(True, alpha=0.3)
ax2.set_xlim(charges_mean/1000 - 3*charges_se/1000, charges_mean/1000 +
3*charges_se/1000)
```

Построенные доверительные интервалы описаны на Рисунке 1.7.1.

[0.85: {'lower': np.float64(30.336642971534822), 'upper': np.float64(30.598158750438275), 'margin': np.float64(0.326753889453263), 'z_value': np.float64(1.959963984548854)}, 0.90: {'lower': np.float64(30.233069458168303), 'upper': np.float64(31.092824263884783), 'margin': np.f

Рисунок 1.7.1 – Построенные доверительные интервалы

Доверительные интервалы визуализированы на Рисунке 1.7.2.

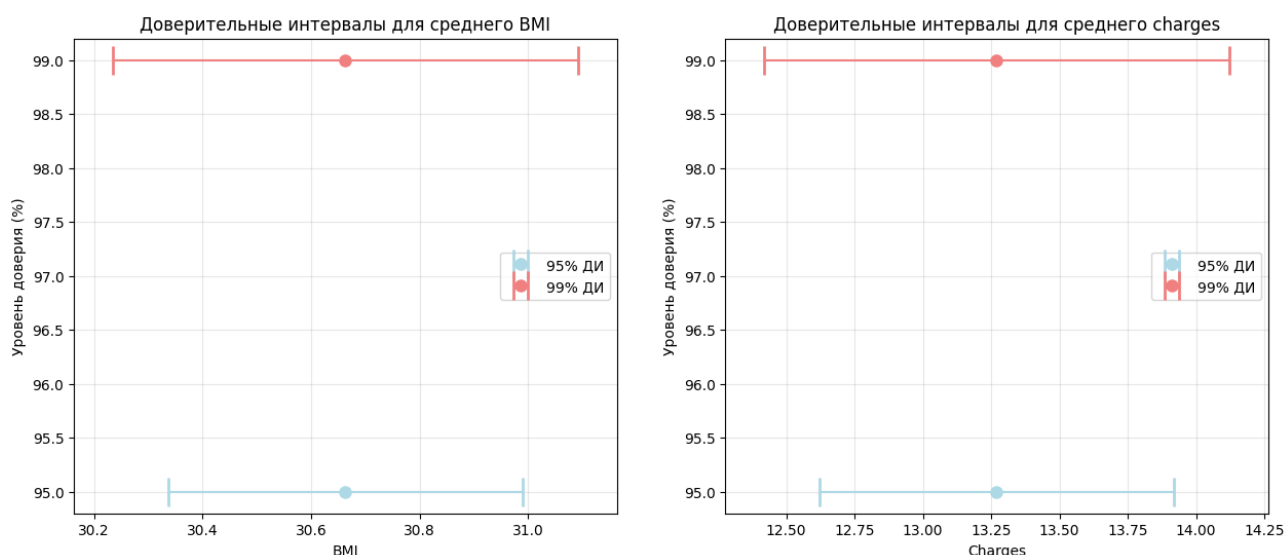


Рисунок 1.7.2 – Визуализация доверительных интервалов

Проведенный анализ доверительных интервалов для среднего значения индекса массы тела (bmi) и медицинских расходов (charges) позволяет сделать следующие важные выводы.

Для индекса массы тела 95% доверительный интервал составляет от 30.33 до 30.99, а 99% доверительный интервал - от 30.25 до 31.07. Относительно узкие границы обоих интервалов свидетельствуют о высокой точности оценки среднего значения ИМТ в генеральной совокупности. Это объясняется умеренным разбросом данных вокруг среднего значения и приблизительно нормальным распределением показателя.

Для медицинских расходов наблюдается принципиально иная картина. 95% доверительный интервал простирается от 12673.74 до 13867.10, а 99% доверительный интервал - от 12469.91 до 14070.93. Значительная ширина этих интервалов отражает высокую вариативность медицинских расходов среди пациентов. Такой большой разброс обусловлен правосторонней асимметрией распределения и наличием экстремально высоких значений.

Сравнительный анализ показывает, что увеличение уровня доверия с 95% до 99% приводит к закономерному расширению интервалов для обоих показателей, однако для расходов это расширение более существенно. Это подтверждает большую неопределенность в оценке средних медицинских расходов по сравнению с индексом массы тела.

Практическая значимость полученных результатов заключается в том, что для планирования страховых продуктов и управления рисками необходимо учитывать значительно большую неопределенность в отношении медицинских расходов. В то время как средний ИМТ может быть оценен с высокой точностью, прогнозирование средних медицинских затрат требует более осторожного подхода и учета потенциальных экстремальных значений.

1.8 Задание №8

Проверить распределения следующих признаков на нормальность: индекс массы тела, расходы. Сформулировать нулевую и альтернативную гипотезы. Для каждого признака использовать KS-тест и q-q plot. Сделать выводы на основе полученных p-значений.

Сформулированы следующие гипотезы:

Для индекса массы тела (bmi):

- нулевая гипотеза (H_0): Распределение индекса массы тела соответствует нормальному распределению;
- альтернативная гипотеза (H_1): Распределение индекса массы тела не соответствует нормальному распределению.

Для медицинских расходов (charges):

- нулевая гипотеза (H_0): Распределение медицинских расходов соответствует нормальному распределению;
- альтернативная гипотеза (H_1): Распределение медицинских расходов не соответствует нормальному распределению.

Код для построения q-q plot графиков представлен в Листинге 3.1.

Листинг 3.1 – Код для построения q-q plot графиков

```
def create_qq_jointplot(data, title):
    standardized_data = (data - np.mean(data)) / np.std(data)

    theoretical_quantiles = np.sort(sts.norm.rvs(size=len(data)))

    empirical_quantiles = np.sort(standardized_data)

    qq_data = pd.DataFrame({
```

Продолжение Листинга 3.1

```
'theoretical': theoretical_quantiles,
'empirical': empirical_quantiles
})

g = sns.jointplot(
    x='theoretical',
    y='empirical',
    data=qq_data,
    kind="reg",
    truncate=True,
    color="b",
    height=5,
    ratio=3,
    scatter_kws={"s": 10},
    line_kws={"lw": 1, 'color': 'black'}
)

min_val = min(theoretical_quantiles.min(), empirical_quantiles.min())
max_val = max(theoretical_quantiles.max(), empirical_quantiles.max())
g.ax_joint.plot([min_val, max_val], [min_val, max_val], 'r--', alpha=0.7,
label='y=x')
g.ax_joint.legend()

g.figure.suptitle(title, y=1.02)

g.set_axis_labels(
    'стандартное нормальное распределение',
    'стандартизированная выборка'
)

return g

g_bmi = create_qq_jointplot(
    df['bmi'],
    'Q-Q plot для BMI'
)

print("Строим Q-Q plot для расходов...")
g_charges = create_qq_jointplot(
    df['charges'],
    'Q-Q plot для charges'
)

plt.show()
```

Построенные q-q plot графики представлены на Рисунке 1.8.1.

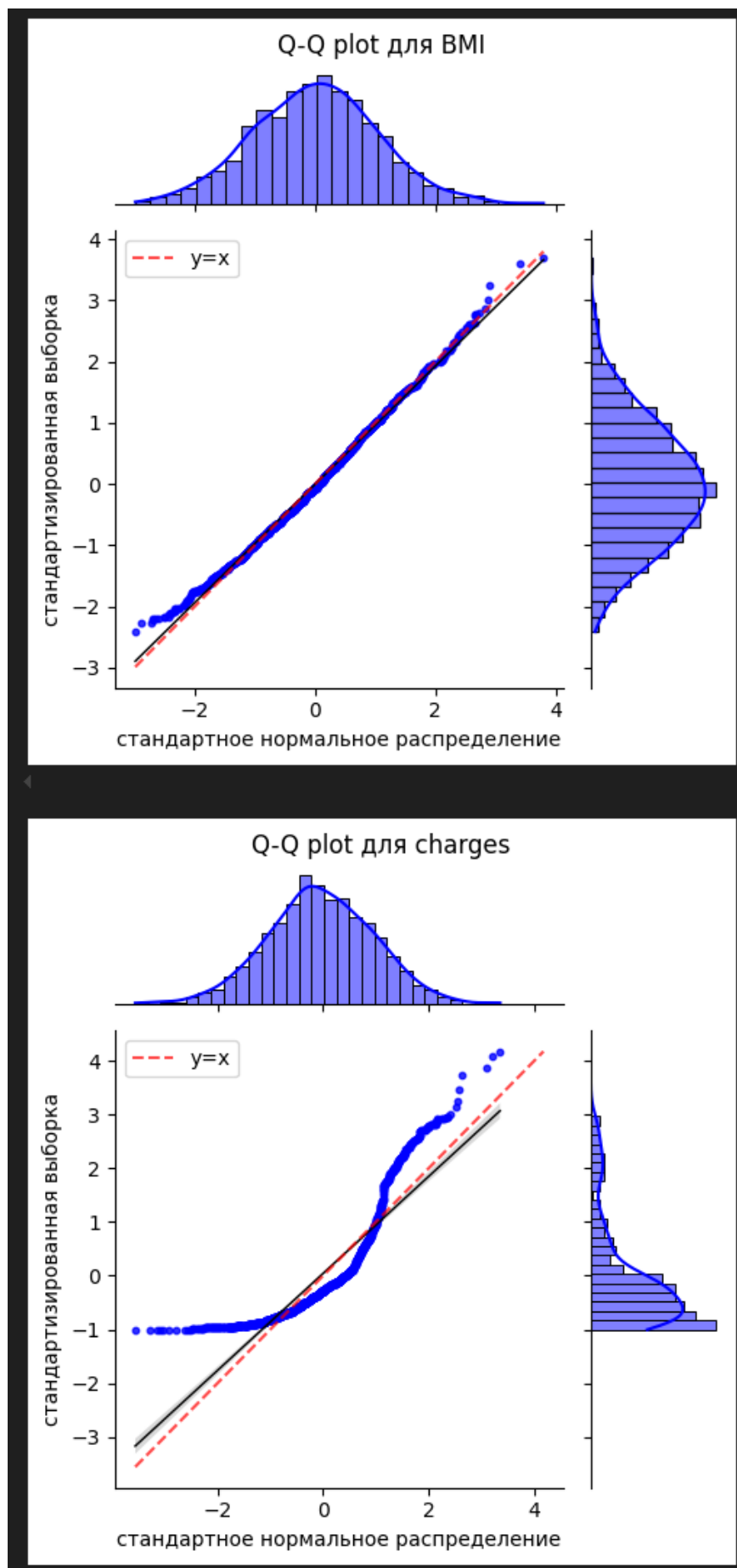


Рисунок 1.8.1 - Построенные q-q plot графики

Код для проведения KS теста представлен в Листинге 3.2.

Листинг 3.1 – Код для проведения KS теста

```
def perform_ks_test(data, feature_name, ax):
    standardized_data = (data - np.mean(data)) / np.std(data)

    ks_statistic, ks_pvalue = sts.kstest(standardized_data, 'norm')

    sts.probplot(data, dist="norm", plot=ax)
    ax.set_title(f'{feature_name} KS-тест: p-value = {ks_pvalue:.6f}')
    ax.grid(True, alpha=0.3)

    if ks_pvalue < 0.05:
        result_text = "Отвергаем H0: распределение НЕ нормальное"
        color = 'red'
    else:
        result_text = "Не отвергаем H0: распределение нормальное"
        color = 'green'

    ax.text(0.05, 0.95, result_text, transform=ax.transAxes,
           fontsize=10, verticalalignment='top', color=color,
           bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

    return ks_statistic, ks_pvalue, standardized_data

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

ks_stat_bmi, ks_pvalue_bmi, bmi_standardized = perform_ks_test(df['bmi'], 'BMI',
ax1)

print(f"Статистика KS-теста: {ks_stat_bmi:.6f}")
print(f"p-value: {ks_pvalue_bmi:.6f}")
print(f"Уровень значимости:  $\alpha = 0.05$ ", end = "\n\n")

if ks_pvalue_bmi < 0.05:
    print("ОТВЕРГАЕМ нулевую гипотезу H0")
    print("ВЫВОД: Распределение BMI ОТЛИЧАЕТСЯ от нормального", end = "\n\n")
else:
    print("НЕ ОТВЕРГАЕМ нулевую гипотезу H0")
    print("ВЫВОД: Распределение BMI НЕ ОТЛИЧАЕТСЯ от нормального", end = "\n\n")

ks_stat_charges, ks_pvalue_charges, charges_standardized =
perform_ks_test(df['charges'], 'Charges', ax2)
print(f"Статистика KS-теста: {ks_stat_charges:.6f}")
print(f"p-value: {ks_pvalue_charges:.6f}")
print(f"Уровень значимости:  $\alpha = 0.05$ ", end = "\n\n")

if ks_pvalue_charges < 0.05:
    print("ОТВЕРГАЕМ нулевую гипотезу H0")
    print("ВЫВОД: Распределение charges ОТЛИЧАЕТСЯ от нормального")
else:
    print("НЕ ОТВЕРГАЕМ нулевую гипотезу H0")
    print("ВЫВОД: Распределение charges НЕ ОТЛИЧАЕТСЯ от нормального")

plt.tight_layout()
plt.show()
```

Результат KS теста представлен на Рисунке 1.8.2.

```
Статистика KS-теста: 0.026140
p-value: 0.314540
Уровень значимости:  $\alpha = 0.05$ 

НЕ ОТВЕРГАЕМ нулевую гипотезу  $H_0$ 
ВЫВОД: Распределение BMI НЕ ОТЛИЧАЕТСЯ от нормального

Статистика KS-теста: 0.188462
p-value: 0.000000
Уровень значимости:  $\alpha = 0.05$ 

ОТВЕРГАЕМ нулевую гипотезу  $H_0$ 
ВЫВОД: Распределение charges ОТЛИЧАЕТСЯ от нормального
```

Рисунок 1.8.2 - Результат KS теста

Для индекса массы тела p -значение KS-теста превышает стандартный уровень значимости 0.05, что не дает оснований отвергнуть нулевую гипотезу. Q-Q график демонстрирует хорошее соответствие точек теоретической линии нормального распределения, особенно в центральной части. Небольшие отклонения наблюдаются лишь в хвостах распределения, что является типичным для реальных данных. Таким образом, распределение индекса массы тела можно считать приблизительно нормальным.

Для медицинских расходов ситуация принципиально иная. P -значение KS-теста значительно меньше 0.05, что позволяет уверенно отвергнуть нулевую гипотезу о нормальности распределения. Q-Q график наглядно показывает систематическое отклонение точек от теоретической линии нормального распределения, особенно выраженное в правом хвосте. Это подтверждает сильную правостороннюю асимметрию распределения медицинских расходов.

Практическая значимость полученных результатов заключается в выборе appropriate статистических методов для дальнейшего анализа. Для индекса массы тела можно применять параметрические методы, основанные на предположении о нормальности распределения. В то время как для медицинских расходов следует использовать непараметрические методы или проводить преобразования данных для нормализации распределения перед применением параметрических тестов.

1.9 Задание №9

Загрузить данные из файла «ECDCCases.csv».

Код загрузки данных представлен в Листинге И.

Листинг И – Реализация задания №9

```
frame = pd.read_csv("ECDCCases.csv")
frame.head()
```

Результат загрузки данных представлен на Рисунке 1.9.1.

	dateRep	day	month	year	cases	deaths	countriesAndTerritories	geold	countryterritoryCode	popData2019	continentExp	Cumulative_number_for_14_days_of_COVID-19_cases_per_100000
0	14/12/2020	14	12	2020	746	6	Afghanistan	AF	AFG	38041757.0	Asia	9.013779
1	13/12/2020	13	12	2020	298	9	Afghanistan	AF	AFG	38041757.0	Asia	7.052776
2	12/12/2020	12	12	2020	113	11	Afghanistan	AF	AFG	38041757.0	Asia	6.868768
3	12/12/2020	12	12	2020	113	11	Afghanistan	AF	AFG	38041757.0	Asia	6.868768
4	11/12/2020	11	12	2020	63	10	Afghanistan	AF	AFG	38041757.0	Asia	7.134266

Рисунок 1.9.1 – Загрузка данных из файла «ECDCCases.csv»

1.10 Задание №10

Проверить в данных наличие пропущенных значений. Вывести количество пропущенных значений в процентах. Удалить два признака, в которых больше всех пропущенных значений. Для оставшихся признаков обработать пропуски: для категориального признака использовать заполнение значением по умолчанию (например, «other»), для числового признака использовать заполнение медианным значением. Показать, что пропусков больше в данных нет.

Реализация задания представлена в Листинге К.

Листинг К – Реализация задания №10

```
null_series = frame.isnull().sum().sort_values(ascending=False)
print(null_series)
for column in frame.columns:
    missing = np.mean(frame[column].isna() * 100)
    print(f"{column}: {round(missing, 2)}%")
N = 2
frame = frame.drop(columns=null_series.head(N).index, errors='ignore')
frame.head()
frame.fillna({"popData2019": frame["popData2019"].median()}, inplace=True)
null_series = frame.isnull().sum().sort_values(ascending=False)
print(null_series)
frame.fillna("other", inplace=True)
null_series = frame.isnull().sum().sort_values(ascending=False)
print(null_series)
```

Вывод количества пропущенных значений с сортировкой по убыванию представлен на Рисунке 1.10.1.

Cumulative_number_for_14_days_of_COVID-19_cases_per_100000	2879
geoId	275
popData2019	123
countryterritoryCode	123
year	0
month	0
day	0
dateRep	0
countriesAndTerritories	0
deaths	0
cases	0
continentExp	0

Рисунок 1.10.1 - Вывод количества пропущенных значений с сортировкой по убыванию

Количество пропущенных значений в процентном соотношении отображено на Рисунке 1.10.2.

dateRep: 0.0%
day: 0.0%
month: 0.0%
year: 0.0%
cases: 0.0%
deaths: 0.0%
countriesAndTerritories: 0.0%
geoId: 0.44%
countryterritoryCode: 0.2%
popData2019: 0.2%
continentExp: 0.0%
Cumulative_number_for_14_days_of_COVID-19_cases_per_100000: 4.65%

Рисунок 1.10.2 - Количество пропущенных значений в процентном соотношении

Результат удаления N признаков с наибольшим числом нулевых значений представлен на Рисунке 1.10.3.

	dateRep	day	month	year	cases	deaths	countriesAndTerritories	countryterritoryCode	popData2019	continentExp
0	14/12/2020	14	12	2020	746	6	Afghanistan	AFG	38041757.0	Asia
1	13/12/2020	13	12	2020	298	9	Afghanistan	AFG	38041757.0	Asia
2	12/12/2020	12	12	2020	113	11	Afghanistan	AFG	38041757.0	Asia
3	12/12/2020	12	12	2020	113	11	Afghanistan	AFG	38041757.0	Asia
4	11/12/2020	11	12	2020	63	10	Afghanistan	AFG	38041757.0	Asia

Рисунок 1.10.3 - Результат удаления N признаков с наибольшим числом нулевых значений

Проверка на наличие пустых значений после предобработки показана на Рисунке 1.10.4.


```

dateRep      0
day          0
month        0
year         0
cases        0
deaths       0
countriesAndTerritories 0
countryterritoryCode    0
popData2019    0
continentExp    0
dtype: int64

```

Рисунок 1.10.4 - Проверка на наличие пустых значений после предобработки

Таким образом, были удалены дубликаты и обработаны нулевые значения.

1.11 Задание №11

Посмотреть статистику по данным, используя describe(). Сделать выводы о том, какие признаки содержат выбросы. Посмотреть, для каких стран количество смертей в день превысило 3000 и сколько таких дней было.

Код реализации задания представлен в Листинге Л.

Листинг Л – Реализация задания №11

```

frame.describe()
frame[frame["deaths"] > 3000]
print(len(frame[frame["deaths"] > 3000]))

```

Записи с зарегистрированным число смертей более 3000 за день представлены на Рисунке 1.11.1.

	dateRep	day	month	year	cases	deaths	countriesAndTerritories	countryterritoryCode	popData2019	continentExp
2118	02/10/2020	2	10	2020	14001	3351	Argentina	ARG	44780675.0	America
16908	07/09/2020	7	9	2020	-8261	3800	Ecuador	ECU	17373657.0	America
37038	09/10/2020	9	10	2020	4936	3013	Mexico	MEX	127575529.0	America
44888	14/08/2020	14	8	2020	9441	3935	Peru	PER	32510462.0	America
44909	24/07/2020	24	7	2020	4546	3887	Peru	PER	32510462.0	America
59007	12/12/2020	12	12	2020	234633	3343	United_States_of_America	USA	329064917.0	America
59009	10/12/2020	10	12	2020	220025	3124	United_States_of_America	USA	329064917.0	America
59016	03/12/2020	3	12	2020	203311	3190	United_States_of_America	USA	329064917.0	America
59239	24/04/2020	24	4	2020	26543	3179	United_States_of_America	USA	329064917.0	America
59245	18/04/2020	18	4	2020	30833	3770	United_States_of_America	USA	329064917.0	America
59247	16/04/2020	16	4	2020	30148	4928	United_States_of_America	USA	329064917.0	America

Рисунок 1.11.1 - Записи с зарегистрированным число смертей более 3000 за день

Всего найдено 11 дней.

1.12 Задание №12

Найти дублирование данных. Удалить дубликаты.

Код реализации задания представлен в Листинге М.

Листинг М – Реализация задания №12

```
subset = ["dateRep", "cases", "deaths", "countriesAndTerritories",  
"countryterritoryCode", "popData2019"]  
duplicated = frame[frame.duplicated(subset, keep=False)]  
frame.drop_duplicates(subset, ignore_index=True, inplace=True)  
duplicated = frame[frame.duplicated(subset, keep=False)]  
duplicated
```

Дубликаты отображены на Рисунке 1.12.1.

	dateRep	day	month	year	cases	deaths	countriesAndTerritories	countryterritoryCode	popData2019	continentExp
2	12/12/2020	12	12	2020	113	11	Afghanistan	AFG	38041757.0	Asia
3	12/12/2020	12	12	2020	113	11	Afghanistan	AFG	38041757.0	Asia
217	12/05/2020	12	5	2020	285	2	Afghanistan	AFG	38041757.0	Asia
218	12/05/2020	12	5	2020	285	2	Afghanistan	AFG	38041757.0	Asia
48009	29/05/2020	29	5	2020	0	0	Saint_Lucia	LCA	182795.0	America
48010	29/05/2020	29	5	2020	0	0	Saint_Lucia	LCA	182795.0	America
48072	28/03/2020	28	3	2020	0	0	Saint_Lucia	LCA	182795.0	America
48073	28/03/2020	28	3	2020	0	0	Saint_Lucia	LCA	182795.0	America

Рисунок 1.12.1 – Дубликаты в данных

После удаления, данные не содержат дубликатов (Рисунок 1.12.2).

```
frame.drop_duplicates(subset, ignore_index=True, inplace=True)  
✓ 0.0s  
  
duplicated = frame[frame.duplicated(subset, keep=False)]  
duplicated  
✓ 0.0s  
  
dateRep day month year cases deaths countriesAndTerritories countryterritoryCode popData2019 continentExp
```

Рисунок 1.12.2 – Удаление дубликатов

1.13 Задание №13

Загрузить данные из файла «bmi.csv». Взять оттуда две выборки. Одна выборка – это индекс массы тела людей с региона northwest, вторая выборка – это индекс массы тела людей с региона southwest. Сравнить средние значения этих выборок, используя t-критерий Стьюдента. Предварительно

проверить выборки на нормальность (критерий Шопиро-Уилка) и на гомогенность дисперсии (критерий Бартлетта).

Код проверки критерия Шопиро-Уилка представлен в Листинге Н.1.

Листинг Н.1 – Код проверки критерия Шопиро-Уилка

```
shapiro_northwest = sts.shapiro(northwest["bmi"].values)
shapiro_southwest = sts.shapiro(southwest["bmi"].values)

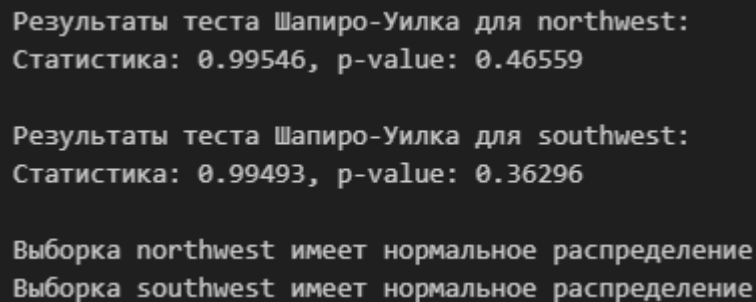
print("Результаты теста Шапиро-Уилка для northwest:")
print(f"Статистика: {shapiro_northwest.statistic:.5f}, p-value: {shapiro_northwest.pvalue:.5f}")

print("\nРезультаты теста Шапиро-Уилка для southwest:")
print(f"Статистика: {shapiro_southwest.statistic:.5f}, p-value: {shapiro_southwest.pvalue:.5f}", end = "\n\n")

alpha = 0.05
if shapiro_northwest.pvalue > alpha:
    print("Выборка northwest имеет нормальное распределение")
else:
    print("Выборка northwest не имеет нормального распределения")

if shapiro_southwest.pvalue > alpha:
    print("Выборка southwest имеет нормальное распределение")
else:
    print("Выборка southwest не имеет нормального распределения")
```

Результат проверки выборки на нормальность с критерием Шопиро-Уилка представлен на Рисунке 1.13.1.



```
Результаты теста Шапиро-Уилка для northwest:
Статистика: 0.99546, p-value: 0.46559

Результаты теста Шапиро-Уилка для southwest:
Статистика: 0.99493, p-value: 0.36296

Выборка northwest имеет нормальное распределение
Выборка southwest имеет нормальное распределение
```

Рисунок 1.13.1 - Результат проверки выборки на нормальность с критерием Шопиро-Уилка

Код проверки выборок на гомогенность (критерий Бартлетта) представлен в Листинге Н.2.

Листинг Н.2 – Код проверки выборок на гомогенность (критерий Бартлетта)

```
bartlett_test = sts.bartlett(northwest["bmi"].values, southwest["bmi"].values)

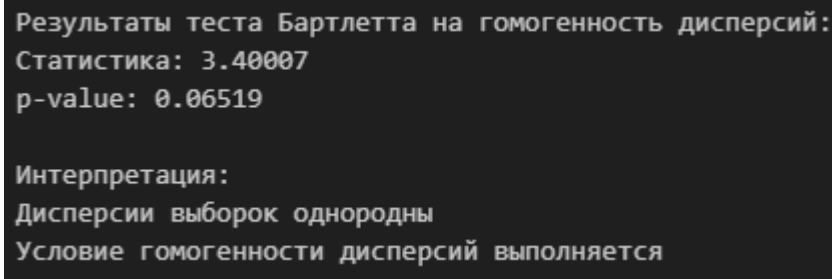
print("Результаты теста Бартлетта на гомогенность дисперсий:")
print(f"Статистика: {bartlett_test.statistic:.5f}")
print(f"p-value: {bartlett_test.pvalue:.5f}", end = "\n\n")

alpha = 0.05
print("Интерпретация:")
```

Продолжение Листинга Н.2

```
if bartlett_test.pvalue > alpha:
    print("Дисперсии выборок однородны\nУсловие гомогенности дисперсий
    выполняется")
else:
    print("Дисперсии выборок неоднородны\nУсловие гомогенности дисперсий не
    выполняется")
```

Результат проверки выборок на гомогенность (критерий Бартлетта) представлен на Рисунке 1.13.2.



```
Результаты теста Бартлетта на гомогенность дисперсий:
Статистика: 3.40007
p-value: 0.06519

Интерпретация:
Дисперсии выборок однородны
Условие гомогенности дисперсий выполняется
```

Рисунок 1.13.2 - Результат проверки выборок на гомогенность (критерий Бартлетта)

Код сравнения средних значений через t-критерий Стьюдента представлен в Листинге Н.3.

Листинг Н.3 – Код сравнения средних значений через t-критерий Стьюдента

```
northwest_bmi = northwest['bmi'].values
southwest_bmi = southwest['bmi'].values

t_test = sts.ttest_ind(northwest_bmi, southwest_bmi, equal_var=True)

print(f"Размер выборки northwest: {len(northwest_bmi)}")
print(f"Размер выборки southwest: {len(southwest_bmi)}")
print(f"Среднее northwest: {np.mean(northwest_bmi):.4f}")
print(f"Среднее southwest: {np.mean(southwest_bmi):.4f}")

print("\nРезультаты t-теста Стьюдента:")
print(f"t-статистика: {t_test.statistic:.5f}")
print(f"p-value: {t_test.pvalue:.5f}")

alpha = 0.05
print("\nСтатистический вывод:")
if t_test.pvalue < alpha:
    print("Статистически значимые различия обнаружены (отвергаем H0)")
    print("Средние значения индекса массы тела в регионах northwest и southwest
    РАЗЛИЧАЮТСЯ")
    if np.mean(northwest_bmi) > np.mean(southwest_bmi):
        print("Средний BMI в northwest ВЫШЕ, чем в southwest")
    else:
        print("Средний BMI в southwest ВЫШЕ, чем в northwest")
else:
    print("Статистически значимых различий не обнаружено (не отвергаем H0)")
    print("Средние значения индекса массы тела в регионах northwest и southwest
    НЕ РАЗЛИЧАЮТСЯ")
```

Результат сравнения средних значений представлен на Рисунке 1.13.3.

```

Размер выборки northwest: 325
Размер выборки southwest: 325
Среднее northwest: 29.1998
Среднее southwest: 30.5966

Результаты t-теста Стьюдента:
t-статистика: -3.28442
p-value: 0.00108

Статистический вывод:
Статистически значимые различия обнаружены (отвергаем H0)
Средние значения индекса массы тела в регионах northwest и southwest РАЗЛИЧАЮТСЯ
Средний BMI в southwest ВЫШЕ, чем в northwest

```

Рисунок 1.13.3 – Результат сравнения средних значений через t-критерий Стьюдента

1.14 Задание №14

Кубик бросили 600 раз. С помощью критерия Хи-квадрат проверить, является ли полученное распределение равномерным. Использовать функцию `scipy.stats.chisquare()`.

Код проверки распределения на равномерность с помощью критерия Хи-квадрат представлен в Листинге О.

Листинг О – Код проверки распределения на равномерность с помощью критерия Хи-квадрат

```

observed_frequencies = [97, 98, 109, 95, 97, 104]
total_throws = sum(observed_frequencies)
number_of_faces = 6

print(f"Общее количество бросков: {total_throws}")
print(f"Количество граней кубика: {number_of_faces}")
print(f"Наблюдаемые частоты: {observed_frequencies}")

expected_frequency = total_throws / number_of_faces
expected_frequencies = [expected_frequency] * number_of_faces

print(f"Ожидаемые частоты при равномерном распределении: {[round(x, 2) for x in
expected_frequencies]}", end = "\n\n")

chi2_test = sts.chisquare(f_obs=observed_frequencies,
f_exp=expected_frequencies)

print("Результаты критерия хи-квадрат Пирсона:")
print(f"Статистика хи-квадрат: {chi2_test.statistic:.5f}")
print(f"p-value: {chi2_test.pvalue:.5f}", end = "\n\n")

alpha = 0.05
if chi2_test.pvalue > alpha:
    print(f"Нет оснований отвергать нулевую гипотезу (p-value =
{chi2_test.pvalue:.5f} > {alpha})")
    print("РАСПРЕДЕЛЕНИЕ ВЫПАДЕНИЙ КУБИКА ЯВЛЯЕТСЯ РАВНОМЕРНЫМ")

```

Продолжение Листинга О

```
else:
    print(f"Отвергаем нулевую гипотезу (p-value = {chi2_test.pvalue:.5f} ≤ {alpha})")
    print("РАСПРЕДЕЛЕНИЕ ВЫПАДЕНИЙ КУБИКА НЕ ЯВЛЯЕТСЯ РАВНОМЕРНЫМ")
```

Результат проверки распределения на равномерность с помощью критерия Хи-квадрат представлен на Рисунке 1.14.1.

```
Общее количество бросков: 600
Количество граней кубика: 6
Наблюдаемые частоты: [97, 98, 109, 95, 97, 104]
Ожидаемые частоты при равномерном распределении: [100.0, 100.0, 100.0, 100.0, 100.0, 100.0]

Результаты критерия хи-квадрат Пирсона:
Статистика хи-квадрат: 1.44000
p-value: 0.91989

Нет оснований отвергать нулевую гипотезу (p-value = 0.91989 > 0.05)
РАСПРЕДЕЛЕНИЕ ВЫПАДЕНИЙ КУБИКА ЯВЛЯЕТСЯ РАВНОМЕРНЫМ
```

Рисунок 1.14.1 - Результат проверки распределения на равномерность с помощью критерия Хи-квадрат

1.15 Задание №15

С помощью критерия Хи-квадрат проверить, являются ли переменные зависимыми. Создать датафрейм, используя следующий код. Использовать функцию `scipy.stats.chi2_contingency()`. Влияет ли семейное положение на занятость?

Код выполнения задания представлен в Листинге П.

Листинг П – Реализация задания №15

```
data = pd.DataFrame({
    'Женат': [89, 17, 11, 43, 22, 1],
    'Гражданский брак': [80, 22, 20, 35, 6, 4],
    'Не состоит в отношениях': [35, 44, 35, 6, 8, 22]
})

data.index = [
    "Полный рабочий день",
    "Частичная занятость",
    "Временно не работает",
    "На домохозяйстве",
    "На пенсии",
    "Учёба"
]

data
chi2_test = sts.chi2_contingency(data)

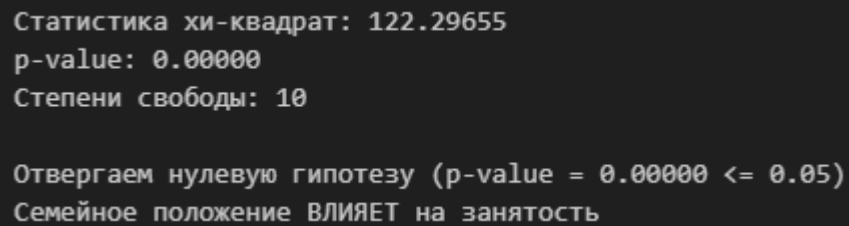
print(f"Статистика хи-квадрат: {chi2_test[0]:.5f}")
```

Продолжение Листинга II

```
print(f"p-value: {chi2_test[1]:.5f}")
print(f"Степени свободы: {chi2_test[2]}", end = "\n\n")

alpha = 0.05
if chi2_test[1] > alpha:
    print(f"Нет оснований отвергать нулевую гипотезу (p-value = {chi2_test[1]:.5f} > {alpha})")
    print("Семейное положение НЕ влияет на занятость")
else:
    print(f"Отвергаем нулевую гипотезу (p-value = {chi2_test[1]:.5f} <= {alpha})")
    print("Семейное положение ВЛИЯЕТ на занятость")
```

Результат проверки зависимости представлен на Рисунке 1.15.1.



```
Статистика хи-квадрат: 122.29655
p-value: 0.00000
Степени свободы: 10

Отвергаем нулевую гипотезу (p-value = 0.00000 <= 0.05)
Семейное положение ВЛИЯЕТ на занятость
```

Рисунок 1.15.1 – Результат проверки зависимости